



BOOTCAMP FRONTEND & REACT · SEMANA 2

CSS Grid

Clase 4 — Colocar, alinear y dibujar el plano de la página · La tienda gana su barra lateral

L·M·V

Proyecto TechCart

STEERCLAVE

LO QUE CUBRIMOS HOY (CON UN RECESO DE 15 MIN)

Agenda

1. Repaso del arranque de Grid

display: grid, columnas, fr, repeat, minmax · auto-fill vs auto-fit en vivo

2. Vocabulario y cuadrícula implícita

líneas, pistas, celdas, áreas · grid-auto-rows, grid-auto-flow · filas a mano

3. Colocar por líneas

grid-column, grid-row, span, la línea -1 · el overlay de DevTools · la destacada 2×2

4. El plano de la página

grid-template-areas, grid-area · la barra lateral · el pie pegado abajo

5. Alinear dentro de la cuadrícula

justify/align · items, self, content · place-items

6. Grid anidado

el formulario en dos columnas · grid-column: 1 / -1

7. Grid o Flexbox · detalles · errores

cuándo cada uno · grid-template, order, subgrid, minmax(0, 1fr) · errores frecuentes

8. Arranque de Responsive y cierre

la primera media query · resumen, tareas y anticipo

Lo que dejó la Clase 3

```
.productos {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(200px, 1fr));
  gap: 16px;
}
```

display: grid

el contenedor es cuadrícula; sus hijos directos, ítems

fr · repeat() · minmax()

fracción del sobrante · no repetir · mínimo y máximo de una columna

auto-fill · con 2 tarjetas



llena la fila con columnas: las vacías quedan reservadas, las tarjetas conservan su ancho

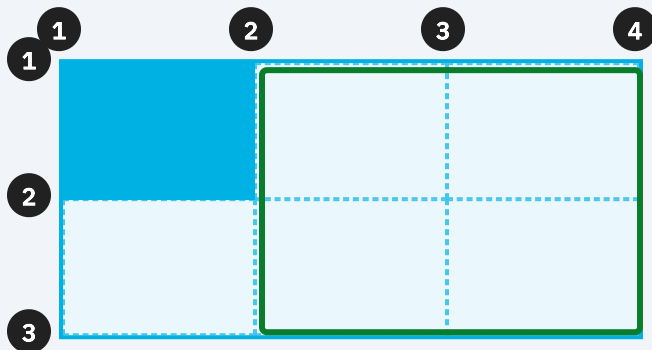
auto-fit · con 2 tarjetas



ajusta las columnas a lo que hay: las vacías colapsan y las tarjetas se estiran

CUATRO PALABRAS PARA HABLAR CON PRECISIÓN

Líneas, pistas, celdas y áreas



■ Líneas

las rayas que separan columnas y filas, incluidas las de los bordes; van **numeradas** (3 columnas → 4 líneas)

■ Pistas (tracks)

el espacio entre dos líneas seguidas: una columna o una fila completa; 1fr 1fr 1fr define tres pistas

■ Celdas

el cruce de una pista de columna con una de fila: donde cae un ítem por defecto

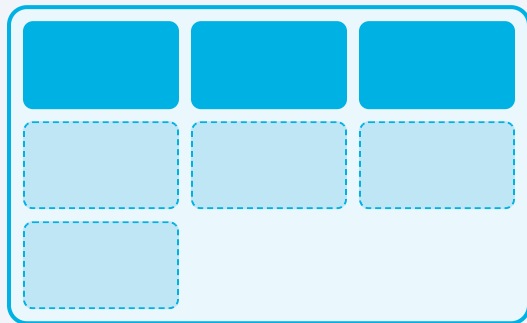
□ Áreas

un rectángulo de una o más celdas; se les puede poner **nombre** (el plano de la página)

¿QUIÉN CREÓ LAS FILAS DEL CATÁLOGO? GRID SOLO

Cuadrícula **explícita** e **implícita**

`grid-template-columns: repeat(3, 1fr)`



explícita · lo que declaro

implícita · la crea Grid

implícita · mide su contenido...

grid-auto-rows

tamaño de las filas que Grid crea solo (1fr: todas iguales). Existe `grid-auto-columns` para el caso contrario.

grid-auto-flow: row | column

en qué orden se rellenan las celdas: fila por fila (fábrica) o columna por columna.

grid-auto-flow: dense

permite **volver atrás** a rellenar huecos; compacta, pero el orden visual deja de ser el del HTML.

MISMO FORMATO QUE LAS COLUMNAS · MISMO VALORES PARA LAS PISTAS

grid-template-rows

lo que acepta una pista (fila o columna)

200px

tamaño fijo

1fr

fracción de lo que sobra

auto

mide lo que mide su contenido

min-content

el mínimo sin desbordar (la palabra más larga)

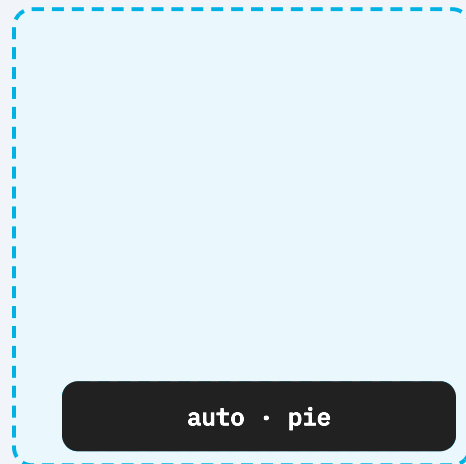
max-content

el máximo (todo en una sola línea)

minmax(200px, 1fr)

entre un mínimo y un máximo

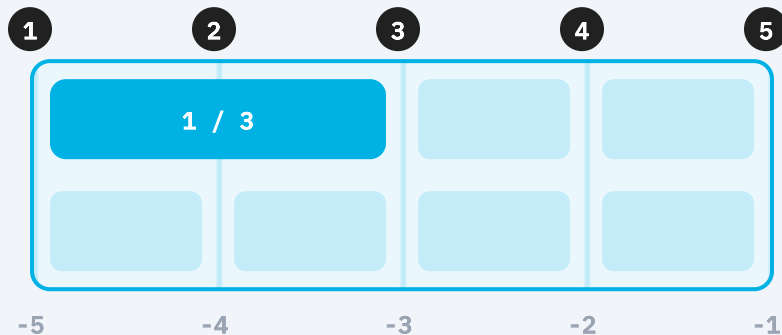
En la práctica, lo que más se escribe es **auto**, **fr** y **minmax()**.

grid-template-rows: auto 1fr auto

hoy: el pie **pegado abajo** (con `min-height: 100vh`)

LA CUADRÍCULA TIENE LÍNEAS, NO SOLO COLUMNAS · SE CUENTAN LOS BARROTES

4 columnas → 5 líneas



grid-column-start: 1 · grid-column-end: 3

en qué línea empieza y en cuál termina

grid-column: 1 / 3

el atajo, con una barra: "de la línea 1 a la línea 3"
(dos columnas)

La primera línea es la 1, no la 0

una reja de 4 espacios tiene **5 barrotes**; los
elementos van de barrote a barrote

Negativos = desde la derecha

-1 es la última: `grid-column: 1 / -1` = de
borde a borde, haya las columnas que haya

grid-row · grid-row-start · grid-row-end

todo lo de columnas existe igual para filas

NO ME CREAN A MÍ: CRÉANLE AL NAVEGADOR

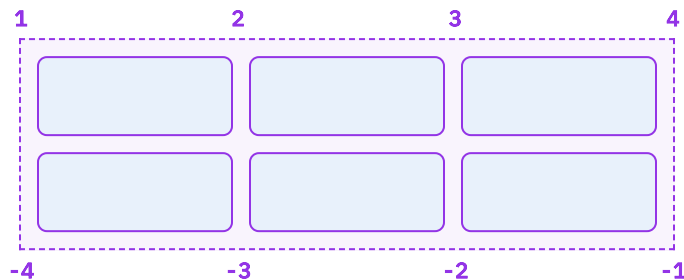
La insignia **grid** y el overlay

Elementos Consola Fuentes

```
<main>
  <section id="catalogo">
    <div class="productos"> grid
    <article class="destacado"> flex
    <article> flex
```

Layout · Grid

- ☒ Mostrar números de línea
- ☒ Mostrar nombres de área
- ☒ Extender líneas de la cuadrícula

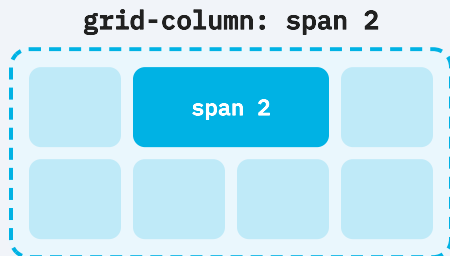


Insignia "grid" junto al contenedor → dibuja la cuadrícula sobre la página, con líneas numeradas (positivas arriba, negativas abajo) y los nombres de las áreas.

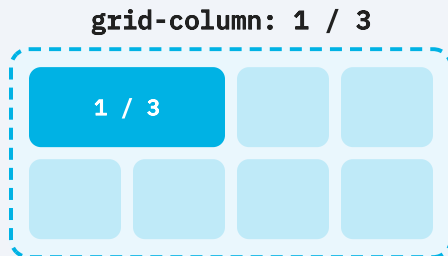
Panel Estilos: cambia `grid-template-columns` a `repeat(2, 1fr)` o `repeat(4, 1fr)` y el overlay se redibuja; F5 y todo vuelve al archivo.

CON LÍNEAS DIGO DÓNDE; CON SPAN DIGO CUÁNTO · LO MÁS USADO

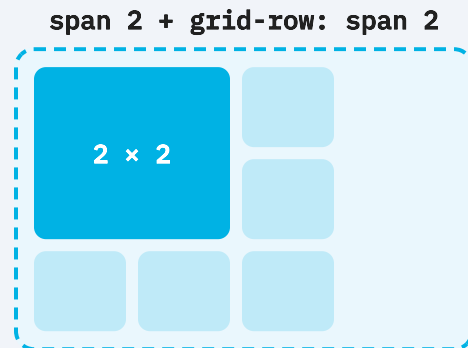
span: ocupar varias pistas



"abarca 2 columnas desde donde caigas"



la forma larga: posición exacta, con líneas



el bloque grande: en TechCart, .destacado

Si el bloque de 2 no cabe al final de una fila, Grid lo baja entero y deja un **hueco**: es `grid-auto-flow: row`, que no vuelve atrás (dense sí, cambiando el orden visual).

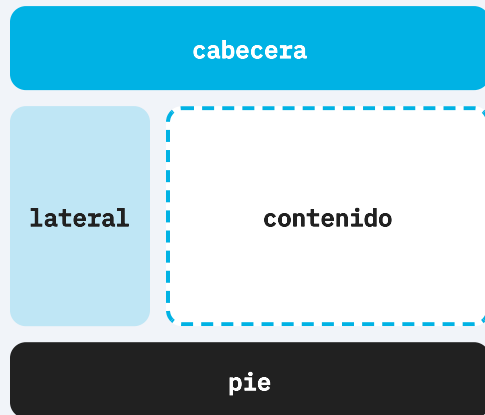
EL LAYOUT DE TODA LA PÁGINA, CON NOMBRES · EL CSS SE LEE COMO UN DIBUJO

grid-template-areas · grid-area

```
body {
  display: grid;
  grid-template-columns: 300px minmax(0, 1fr);
  grid-template-areas:
    "cabecera cabecera"
    "lateral contenido"
    "pie pie";
  gap: 24px;
}
header { grid-area: cabecera; }
aside { grid-area: lateral; }
```

grid-area: 1 / 1 / 3 / 6

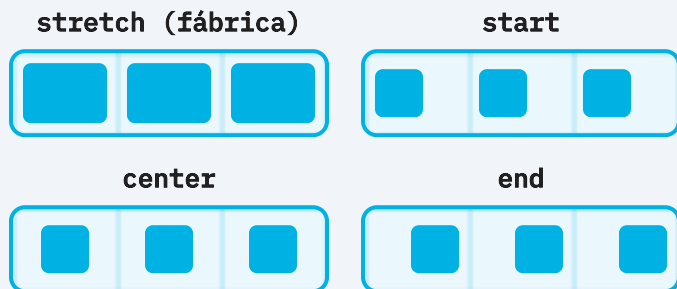
la otra cara de la misma propiedad: fila-inicio / columna-inicio / fila-fin / columna-fin



una cadena por fila · un nombre por columna
todas las filas con la **misma cantidad** de
nombres

CADA ÍTEM VIVE DENTRO DE SU CELDA · POR DEFECTO LA LLENA (STRETCH)

Alinear dentro de la cuadrícula



justify-items · align-items · place-items

todos, cada uno dentro de su celda; va en el contenedor.
place-items: center = los dos ejes

justify-self · align-self

uno solo, la excepción; va en el ítem y manda (la barra lateral: align-self: start)

justify-content · align-content

la **cuadrícula entera** dentro del contenedor, solo si sobra espacio; no mueve ítems dentro de su celda

En Grid los ejes **no rotan**: justify = horizontal · align = vertical.
Siempre.

UN ÍTEM DE UNA CUADRÍCULA PUEDE SER CONTENEDOR DE OTRA

Grid **anidado**: el formulario en dos columnas

- fieldset.envio · display: grid -

```
.envio {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 0 16px;
}

.campo-ancho { grid-column: 1 / -1; }
```

Los ítems son los <p>

cada uno con su etiqueta y su campo; Grid los reparte de a dos por fila

Flexbox sigue adentro de cada <p>

en una celda angosta etiqueta y campo pasan a flex-basis: 100%; etiqueta arriba, campo abajo

Notas: de la línea 1 a la -1

cruza todas las columnas que haya, hoy dos

NO COMPITEN: SE COMBINAN EN LA MISMA PÁGINA

¿Grid o Flexbox?

Flexbox • UNA dimensión

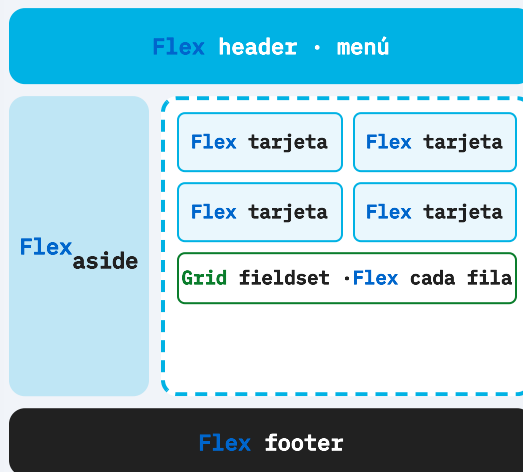


Una fila o una columna; el contenido decide cuánto ocupa cada ítem. "Una hilera de cosas".

Grid • DOS dimensiones



Filas y columnas a la vez; la cuadrícula decide dónde cae cada cosa. "Un plano".



Grid

el plano del body, el catálogo, el fieldset de envío

Flexbox

la cabecera, el menú, cada tarjeta, cada fila del formulario, el pie

Regla

Grid para la estructura, Flexbox para el contenido

CUATRO COSAS QUE NO TECLEAMOS HOY, PERO QUE HAY QUE CONOCER

Detalles que conviene saber

grid-template: auto 1fr auto / 300px 1fr

El **atajo**: filas, una barra, columnas. Es lo mismo que `grid-template-rows` + `grid-template-columns` por separado; lo van a ver en código ajeno.

grid-template-columns: subgrid

El grid anidado **hereda las líneas del padre** en vez de tener las suyas: todo queda alineado con la cuadrícula grande. Funciona en todos los navegadores actuales; solo hay que saber que existe.

order en Grid

Existe, pero **solo afecta a los ítems auto-colocados**. Un ítem con `grid-area` o `grid-column` no le hace caso.

Para mover algo en Grid, se cambia el plano.

1fr = minmax(auto, 1fr)

La columna **nunca baja del ancho de su contenido**: una tabla, una imagen o una palabra larga la ensanchan y la página desborda. Con `minmax(0, 1fr)` puede encoger de verdad. Cuando un grid "se sale", mirar esto primero.

LO QUE MÁS SE ROMPE AL EMPEZAR (Y CÓMO SE ARREGLA)

Errores frecuentes de Grid

1

grid-template-areas con filas de distinto número de columnas → el plano entero se descarta en silencio y todo queda apilado; en DevTools las áreas no aparecen.

2

Comillas mal puestas en el plano → cada fila va entre **sus propias** comillas ("a a" "b c"), no todas las filas dentro de un solo par.

3

Confundir auto-fill con auto-fit → fill reserva columnas vacías (las tarjetas conservan su ancho); fit las colapsa (se estiran). Para un catálogo, fill.

4

1fr sin minmax(0, ...) en una columna con contenido ancho → la columna no encoge y la página desborda en horizontal.

5

display: grid en el ítem en vez del contenedor (o esperar que un nieto sea ítem) → solo los **hijos directos** son ítems. El aside tuvo que salir del main.

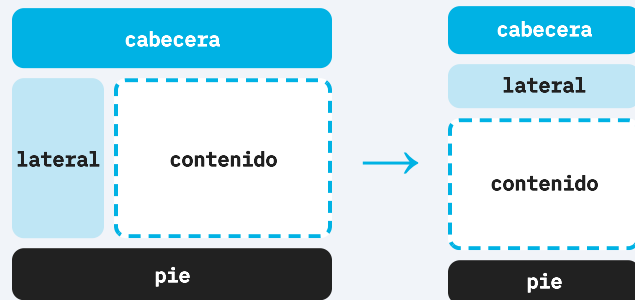
6

Contar las líneas desde 0 → empiezan en **1**; 4 columnas son 5 líneas y la última es la **-1**. La insignia grid de DevTools las muestra.

@media · la misma página en el celular

```
@media (max-width: 600px) {
  body {
    grid-template-columns: minmax(0, 1fr);
    grid-template-areas:
      "cabecera" "lateral"
      "contenido" "pie";
    grid-template-rows: auto auto 1fr auto;
  }
  .destacado { grid-column: auto; grid-row: auto; }
  .envio { grid-template-columns: 1fr; }
  .productos { grid-template-rows: auto; }
}
```

Un bloque que solo se aplica si se cumple la condición: **"600 píxeles o menos"**. Va al final de la hoja: en lo que choque, gana.
Requiere el meta viewport del head (Clase 1).



Resumen

líneas · pistas · celdas · áreas

el vocabulario; explícita (lo que declaro) e implícita (lo que Grid crea):
grid-auto-rows, grid-auto-flow (dense)

grid-template-rows

filas a mano; auto, fr, min/max-content, minmax(); auto 1fr auto + min-height: 100vh = pie pegado abajo

grid-column · grid-row · span · -1

colocar por líneas (4 columnas = 5 líneas, la primera es 1) o por tamaño; 1 / -1 = de borde a borde

grid-template-areas · grid-area

el plano dibujado con nombres: cabecera / lateral + contenido / pie; el aside salió del main

items · self · content · place-items

todos en su celda · uno solo · la cuadrícula entera si sobra espacio; los ejes no rotan

grid anidado

el fieldset de envío en 1fr 1fr; Flexbox sigue adentro de cada fila; notas con 1 / -1

Grid o Flexbox

dos dimensiones para la estructura, una para el contenido; se combinan en capas

grid-template · order · subgrid · minmax(0, 1fr)

el atajo, order solo en auto-colocados, subgrid existe, 1fr puede desbordar

@media (max-width: 600px)

arranque de Responsive: el plano redibujado a una columna; sigue en la Clase 5

Tareas

F

Fácil: replica la clase completa (aside fuera del main, plano, destacada 2×2, lateral, fieldset en dos columnas, media query) y haz que la **segunda tarjeta** también ocupe dos columnas con `grid-column: span 2`. Si aparece un hueco, prueba `grid-auto-flow: dense`. De yapa, **cssgridgarden.com**: 28 niveles de Grid, como el Froggy de la clase pasada.

I1

Intermedia: una zona **"promo"** en el plano: `<section class="promo">` entre header y main, `grid-area: promo`, y la fila `"promo promo"` en el dibujo (también en la media query).

I2

Intermedia: la **lateral a la derecha** sin tocar el HTML: la fila del medio pasa a `"contenido lateral"` y las columnas a `minmax(0, 1fr) 300px`. Compruébalo con la insignia grid.

D1

Difícil: el **pie de una tienda real** con Grid: tres `<div>` (marca, enlaces, contacto) y footer `{ display: grid; grid-template-columns: repeat(3, 1fr); }`; en el celular, una columna.

D2

Difícil: el **<form> entero como grid** de `2fr 1fr`: envío a la izquierda en dos filas (`grid-row: 1 / 3`), pago y términos a la derecha, el botón con `grid-column: 1 / -1` y `justify-self: start`. Investiga el atajo `grid-template`.

Próxima clase

Clase 5 — Responsive completo

Las unidades relativas (**%**, **em**, **rem**, **vw**, **vh**) para que todo escale, más puntos de corte y la forma de pensar **mobile first**, **variables CSS** para la paleta con nombre y el modo oscuro, y **transiciones** para que las tarjetas respondan al mouse.

Pista para casa: abre TechCart en el celular (o achica mucho la ventana) y anota qué se ve apretado o descuadrado. De ahí arrancamos.